KUNJARA OS™ · A PRODUCT OF INDIGO EVENTS

Launch Status Report

The AI-Powered Event Operating System for India

Prepared for **Gautam Shah** · Founder
Session close · **18 April 2026**
Resuming · **Monday, 20 April 2026**

1. Executive Summary

The original target was "launch today" (18 April 2026). That did not land. What did land is the **critical revenue path** and the **hero AI feature**, wired end-to-end through Razorpay and OpenAI. What remains before a real customer can pay and use the product is mostly *validation* rather than new code: the code paths compile clean and pass unauthenticated smoke tests, but no one has clicked through them with a real signed-in session.

Three commits shipped on the `main` branch:

`7a5676c`	**feat(proposals)**: wire AI proposal flow end-to-end · +433 lines, 4 files
`9a1f54f`	**fix(billing)**: make the pay flow actually work end-to-end · +591/−331, 8 files
`77cba33`	**fix**: resolve `/` route conflict and serve marketing site · +7/−319, 6 files

2. What's Finished

These paths are coded, type-check clean, production-build clean, and smoke-tested without auth.

CAPABILITY	STATUS	HOW IT WORKS TODAY
Homepage at `/`	DONE	Redirects to `public/site/index.html` (921-line marketing site matching the blueprint: hero, pillars, ₹1,999–₹24,999 pricing, privacy promise, India flag footer).
Route conflict resolution	DONE	Deleted duplicate `app/(marketing)/` route group. Added `/waitlist` rewrite so pricing CTAs resolve.
Signup and login	DONE	Supabase email-password auth. Signup sends confirmation email. Login redirects to `/dashboard`.
Auth gate on 10 app routes	DONE	`proxy.ts` (Next.js 16 middleware) gates `/dashboard`, `/proposals`, `/budget`, `/compliance`, `/events`, `/vendors`, `/ai`, `/billing`, `/settings`, plus auth-only `/login`, `/signup`.
`/billing` revenue path	DONE	Four canonical plans (`basic` ₹1,999 · `pro` ₹3,999 · `expert` ₹11,999 · `enterprise` ₹24,999). Monthly/annual toggle. Live Supabase usage fetch. Real `create-order → Razorpay Checkout → verify` flow. UPI-first.
Razorpay backend routes	DONE	`POST /api/billing/create-order` (auth-gated, attaches user_id to order notes), `POST /api/billing/verify` (HMAC signature check, server-side plan resolution), `POST /api/billing/webhook` (idempotent, service-role, handles all 4 plans).
Security hardening	DONE	Deleted 3 orphan insecure endpoints: `/api/user/add-credits` (self-grant credits), `/api/billing/verify-payment` (client-specified plan), `/api/billing/capture-payment` (no auth).
AI Proposal Builder — 3-step flow	DONE	`POST /api/proposals/generate-ideas` (GPT-4o, returns 3 scored concepts, no credit burn). `POST /api/proposals/generate-experience` (expands chosen concept to full 7-section `ProposalData`, burns 1 credit).
Proposal list empty state	DONE	`GET /api/proposals` returns `[]` so sidebar click renders "No proposals yet" with CTA to

CAPABILITY	STATUS	HOW IT WORKS TODAY
		`/proposals/new`.
Plan system consolidation	DONE	Removed the mock `lib/billing.ts` with a second, incompatible plan taxonomy (`FREE/STARTER/PRO/ENTERPRISE`). Canonical plans now live in `lib/plans.ts` and match the blueprint.

3. First Thing Monday — Validate Live

These paths compile and route correctly but have **not** been click-tested with a real signed-in user. The risk is that the OpenAI JSON shape or Razorpay credit flow behaves differently in reality than expected.

- **Sign up a test account**. Confirm email. Land on `/dashboard`.
- **Walk `/proposals/new`** end-to-end. Fill the form, generate 3 ideas, pick one, generate full plan. Verify the 7 rich sections render without errors. If `ProposalOutput.tsx` throws on a missing field, iterate the prompt in `app/api/proposals/generate-experience/route.ts`.
- **Walk `/billing`** with a real ₹1 test payment. Watch the Razorpay dashboard: order has `user_id` in notes, webhook fires, `user_usage` row shows correct `credits_added`.
- **Check Supabase schema**. Confirm `user_usage` table exists with columns `user_id, plan, events_used, credits_added, last_payment_id, period_start, period_end, updated_at`. If missing, billing silently fails.

4. Environment & Config Blockers

Two `.env.local` items must be corrected before real customers pay.

Blocker 4.1 · Webhook secret is a placeholder.
`RAZORPAY_WEBHOOK_SECRET=your_razorpay_webhook_secret` — the webhook will return 503 until this is set. Without it, credits only apply via the client-side verify step. If the user closes their browser before verify runs, the payment is captured but credits are not granted — they've paid you with nothing to show.

Fix: Razorpay dashboard → Settings → Webhooks → create webhook pointing to `https://kunjaraos.com/api/billing/webhook` with event `payment.captured`. Copy the generated secret into `.env.local` and into Vercel's env vars.

Blocker 4.2 · Razorpay keys are LIVE, not test.
`RAZORPAY_KEY_ID=rzp_live_SeeYZWNmVe55zF`. Every dev test charges real money. Smoke tests today created a real live order for ₹1,999 (it auto-cancels unpaid so no harm done, but the risk is real).

Fix: Generate test keys in Razorpay dashboard. Use `rzp_test_…` in local dev and staging; keep `rzp_live_…` only in the production Vercel env.

5. What's Left — Prioritised Backlog

Each item is its own scoped piece of work. The rule is **one plan at a time, no bundling**. The numbers reflect priority for a launch, not time estimates.

5.1 · Data correctness (P0 — block launch)

ITEM	STATUS	WHAT TO DO
Usage model mismatch	TO DO	`checkUsage()` in `lib/usage.ts` gates on `events_used >= plan.events`, but billing writes to `credits_added`. Paid credits are stored and never spent; new users get 10 free proposals because `checkUsage` auto-inserts a row defaulting to plan "basic". Rewrite `checkUsage` to read `credits_added` as authoritative. New user default: 0 credits (or 1 free trial credit).
Proposal persistence	TO DO	Generated proposals are shown then lost on refresh. `/proposals/[id]` inaccessible. `GET /api/proposals` returns `[]`. Create a `proposals` table in Supabase (JSONB blob for the full `ProposalData`). Save after `generate-experience`. Implement `GET /api/proposals` + `GET /api/proposals/[id]`.
Supabase schema verification	TO DO	The `user_usage` table schema is unverified. Code expects columns: `user_id, plan, events_used, credits_added, last_payment_id, period_start, period_end, updated_at`. If any column is missing, billing verify/webhook silently fail. Run a dry-run `SELECT` in Supabase SQL editor; add migrations for any missing columns.

5.2 · Features with UI but no backend (P1)

These pages render mock/sample data today. Anyone who clicks a button triggers a request to an endpoint that doesn't exist. `lib/api.ts` references roughly 15 endpoints that aren't implemented.

PAGE	UI	WHAT'S MISSING
`/dashboard`	MOCK ONLY	Shows hardcoded event list. Needs: real event/vendor/compliance stats pulled from DB.
`/events`	MOCK ONLY	`GET /api/events` and `GET /api/events/[id]` missing. No events table.
`/vendors`	MOCK ONLY	`GET/POST /api/vendors` and `PATCH/DELETE /api/vendors/[id]` missing.

PAGE	UI	WHAT'S MISSING
`/compliance`	**CATALOGUE DONE, STATE MOCKED**	11 licence templates already in `lib/compliance.ts`. Need per-event persistence (upload docs, track status, deadline alerts).
`/budget`	**CLIENT-SIDE ONLY**	Budget waterfall works in memory. No save/load. Needs a `budgets` table and `/api/budgets` CRUD.
`/ai`	**SHELL ONLY**	UI exists, backend endpoints `/api/ai/event-plan`, `/api/ai/vendor-suggestions`, `/api/ai/compliance-risk` exist but behaviour unverified.
`/events/[id]/room`	**MOCK** `MOCK_ROOM`	Event Room shell has 8 sub-components (Proposal, Budget, Timeline, Vendors, Tasks, Payments, Approvals). Needs real `event_rooms` table + client-facing branded URL.

5.3 · Blueprint features not started (P2)

- ✗ **Bilingual English/Hindi toggle** — blueprint requires Noto Sans Devanagari + Google Translate API across site and app.
- ✗ **Proposal Intelligence** — track opens, view count, time spent; AI "high intent" signal.
- ✗ **Event microsite builder** — one-click AI-generated event website with agenda, Google Maps, RSVP, auto-expire after event date.
- ✗ **Social Media AI** — caption generation for IG, LinkedIn, FB, YouTube; multi-platform publisher.
- ✗ **Presentation Engine** — 5 deck templates (Concept, Budget, Post-event, Sponsorship, Vendor brief) auto-populated from event data.
- ✗ **Gmail / Drive / WhatsApp integrations** — OAuth flows for sending proposals, file sync, attendee notifications.
- ✗ **CRM pipeline** — leads, stages, follow-up reminders.
- ✗ **Kunjara OS logo** — minimal geometric elephant outline; not yet designed or inserted.
- ✗ **DPDP 90-day deletion** — per blueprint compliance promise. No deletion job exists.

6. How to Execute — The Playbook

6.1 · Rules we learned this sprint

- **One plan at a time.** The codebase already shows what happens when UI ships ahead of backend — `lib/api.ts` accumulated 15+ endpoint references to nothing. Build backend first, UI second.
- **Verify schemas before writing persistence.** The `credits_added` vs `events_used` model mismatch happened because two developers wrote to the same table with different assumptions. Check Supabase dashboard first.
- **Delete orphan endpoints aggressively.** Three insecure routes (`add-credits` , `verify-payment` , `capture-payment`) survived because nothing called them. Any unused endpoint is a liability.
- **Hardcode Supabase creds in the client lib, not env vars.** The repeated "fix supabase env crash" commits in git history prove `process.env` reads at module-load break on Vercel cold starts. String literals in `lib/supabase/client.ts` + `server.ts` side-step this.

6.2 · Monday morning (2 hours)

1. Set `RAZORPAY_WEBHOOK_SECRET` in `.env.local` and in Vercel production env. Swap live keys for test keys in local dev.
2. Open Supabase dashboard → SQL editor → `SELECT * FROM user_usage LIMIT 1;` . Confirm all 8 columns. If missing, add them.
3. Sign up a test user through the live site. Walk `/proposals/new` end-to-end. Screenshot the full proposal output.
4. If the output looks good, make a real ₹1 Razorpay test payment through `/billing` . Confirm `user_usage` row updates, Razorpay dashboard shows webhook fired, and subsequent proposal generation deducts a credit.

6.3 · Monday afternoon — pick one of these

Don't pick more than one; this week is the "go slow" week. Each of these is a full day of focused work.

1. **Fix the usage model** — rewrite `checkUsage` to use `credits_added` as source of truth. This closes the "10 free proposals for free users" loophole. Highest leverage for revenue.
2. **Add proposal persistence** — create `proposals` table, save after generate-experience, wire `GET /api/proposals` and `GET /api/proposals/[id]` . Unblocks the list view and lets users revisit work.
3. **Event Room v1 with real data** — create `event_rooms` table, wire the 8 sub-components. This is the blueprint's flagship differentiator.

6.4 · How to work with Claude

- **One task, not a dump.** "Fix the usage model" gets better output than "build everything in the roadmap".
- **Ask for plan first when the task touches 3+ files.** Plan mode catches the kind of mismatch that caused the billing/usage schema drift.
- **Launch a test user to click through after each Claude session.** Unauth smoke tests caught the ones this sprint didn't catch.

- **Commit in small scopes.** Three commits today, each with a focused message and a concrete Before/After. Makes rollback trivial.

7. Files Touched This Sprint

PATH	CHANGE	COMMIT
`app/page.tsx`	Modified · redirect target	77cba33
`app/(marketing)/*`	Deleted · duplicate route group	77cba33
`next.config.ts`	Modified · `/waitlist` rewrite	77cba33
`app/(app)/billing/page.tsx`	Added · rewritten from mock to real flow	9a1f54f
`app/api/billing/create-order/route.ts`	Modified · require auth, embed user_id	9a1f54f
`app/api/billing/verify/route.ts`	Modified · all 4 plans	9a1f54f
`app/api/billing/webhook/route.ts`	Modified · all 4 plans	9a1f54f
`app/api/billing/capture-payment/`	Deleted · orphan, no auth	9a1f54f
`app/api/billing/verify-payment/`	Deleted · orphan, client-trusted plan	9a1f54f
`app/api/user/add-credits/`	Deleted · orphan, self-grant credits	9a1f54f
`components/ui/RazorpayCheckout.tsx`	Deleted · orphan, never imported	9a1f54f
`lib/billing.ts`	Deleted · orphan second plan system	9a1f54f
`app/api/proposals/generate-ideas/route.ts`	Added · 3 scored concepts	7a5676c
`app/api/proposals/generate-experience/route.ts`	Added · full ProposalData	7a5676c
`app/api/proposals/route.ts`	Modified · added GET returning []	7a5676c
`proxy.ts`	Modified · widen gate from 3 to 10 routes	7a5676c

All three commits are on `main`. No force-pushes. No destructive git operations. The pre-existing in-flight work (dashboard, proposals UI, compliance, budget components, etc.) is untouched and uncommitted — ready for separate focused commits when you're ready.